Sign in

openai / whisper Public

Notifications Fork 9.3k Star 77.8k

Code Pull requests 94 Discussions Actions Security

main whisper / whisper Go to file

/ tokenizer.py

jongwook large-v3 (#1761) c5d4256 · 2 years ago

395 lines (340 loc) · 12 KB

Code Blame Raw

```
import os
import string
from dataclasses import dataclass, field
from functools import cached_property, lru_cache
from typing import Dict, List, Optional, Tuple

import tiktoken

LANGUAGES = {
    "en": "english",
    "zh": "chinese",
    "de": "german",
    "es": "spanish",
```

```
        "ru": "russian",
        "ko": "korean",
        "fr": "french",
        "ja": "japanese",
        "pt": "portuguese",
        "tr": "turkish",
        "pl": "polish",
        "ca": "catalan",
        "nl": "dutch",
        "ar": "arabic",
        "sv": "swedish",
        "it": "italian",
        "id": "indonesian",
        "hi": "hindi",
        "fi": "finnish",
        "vi": "vietnamese",
        "he": "hebrew",
        "uk": "ukrainian",
        "el": "greek",
        "ms": "malay",
        "cs": "czech",
        "ro": "romanian",
        "da": "danish",
        "hu": "hungarian",
        "ta": "tamil",
        "no": "norwegian",
        "th": "thai",
        "ur": "urdu",
        "hr": "croatian",
        "bg": "bulgarian",
        "lt": "lithuanian",
        "la": "latin",
        "mi": "maori",
        "ml": "malayalam",
        "cy": "welsh",
        "sk": "slovak",
        "te": "telugu",
        "fa": "persian",
```

```
        "lv": "latvian",
        "bn": "bengali",
        "sr": "serbian",
        "az": "azerbaijani",
        "sl": "slovenian",
        "kn": "kannada",
        "et": "estonian",
        "mk": "macedonian",
        "br": "breton",
        "eu": "basque",
        "is": "icelandic",
        "hy": "armenian",
        "ne": "nepali",
        "mn": "mongolian",
        "bs": "bosnian",
        "kk": "kazakh",
        "sq": "albanian",
        "sw": "swahili",
        "gl": "galician",
        "mr": "marathi",
        "pa": "punjabi",
        "si": "sinhala",
        "km": "khmer",
        "sn": "shona",
        "yo": "yoruba",
        "so": "somali",
        "af": "afrikaans",
        "oc": "occitan",
        "ka": "georgian",
        "be": "belarusian",
        "tg": "tajik",
        "sd": "sindhi",
        "gu": "gujarati",
        "am": "amharic",
        "yi": "yiddish",
        "lo": "lao",
        "uz": "uzbek",
        "fo": "faroese",
        "ht": "haitian creole"
```

```
        "ht": "haitian creole",
        "ps": "pashto",
        "tk": "turkmen",
        "nn": "nynorsk",
        "mt": "maltese",
        "sa": "sanskrit",
        "lb": "luxembourgish",
        "my": "myanmar",
        "bo": "tibetan",
        "tl": "tagalog",
        "mg": "malagasy",
        "as": "assamese",
        "tt": "tatar",
        "haw": "hawaiian",
        "ln": "lingala",
        "ha": "hausa",
        "ba": "bashkir",
        "jw": "javanese",
        "su": "sundanese",
        "yue": "cantonese",
    }

    # language code lookup by name, with a few language aliases
    TO_LANGUAGE_CODE = {
        **{language: code for code, language in LANGUAGES.items()},
        "burmese": "my",
        "valencian": "ca",
        "flemish": "nl",
        "haitian": "ht",
        "letzeburgesch": "lb",
        "pushto": "ps",
        "panjabi": "pa",
        "moldavian": "ro",
        "moldovan": "ro",
        "sinhalese": "si",
        "castilian": "es",
        "mandarin": "zh",
    }

```

```python

    @dataclass
    class Tokenizer:
        """A thin wrapper around `tiktoken` providing quick access to special tokens"""

        encoding: tiktoken.Encoding
        num_languages: int
        language: Optional[str] = None
        task: Optional[str] = None
        sot_sequence: Tuple[int] = ()
        special_tokens: Dict[str, int] = field(default_factory=dict)

        def __post_init__(self):
            for special in self.encoding.special_tokens_set:
                special_token = self.encoding.encode_single_token(special)
                self.special_tokens[special] = special_token

            sot: int = self.special_tokens["<|startoftranscript|>"]
            translate: int = self.special_tokens["<|translate|>"]
            transcribe: int = self.special_tokens["<|transcribe|>"]

            langs = tuple(LANGUAGES.keys())[: self.num_languages]
            sot_sequence = [sot]
            if self.language is not None:
                sot_sequence.append(sot + 1 + langs.index(self.language))
            if self.task is not None:
                task_token: int = transcribe if self.task == "transcribe" else translate
                sot_sequence.append(task_token)

            self.sot_sequence = tuple(sot_sequence)

        def encode(self, text, **kwargs):
            return self.encoding.encode(text, **kwargs)

        def decode(self, token_ids: List[int], **kwargs) -> str:
            token_ids = [t for t in token_ids if t < self.timestamp_begin]
            return self.encoding.decode(token_ids, **kwargs)

```

```python
    def decode_with_timestamps(self, token_ids: List[int], **kwargs) -> str:
        """
        Timestamp tokens are above other special tokens' id range and are ignored by `decode()`.
        This method decodes given tokens with timestamps tokens annotated, e.g. "<|1.08|>".
        """
        return self.encoding.decode(token_ids, **kwargs)

    @cached_property
    def eot(self) -> int:
        return self.encoding.eot_token

    @cached_property
    def transcribe(self) -> int:
        return self.special_tokens["<|transcribe|>"]

    @cached_property
    def translate(self) -> int:
        return self.special_tokens["<|translate|>"]

    @cached_property
    def sot(self) -> int:
        return self.special_tokens["<|startoftranscript|>"]

    @cached_property
    def sot_lm(self) -> int:
        return self.special_tokens["<|startoflm|>"]

    @cached_property
    def sot_prev(self) -> int:
        return self.special_tokens["<|startofprev|>"]

    @cached_property
    def no_speech(self) -> int:
        return self.special_tokens["<|nospeech|>"]

    @cached_property
    def no_timestamps(self) -> int:
        return self.special_tokens["<|notimestamps|>"]
```

```
207         @cached_property
208         def timestamp_begin(self) -> int:
209             return self.special_tokens["<|0.00|>"]
210 
211         @cached_property
212         def language_token(self) -> int:
213             """Returns the token id corresponding to the value of the `language` field"""
214             if self.language is None:
215                 raise ValueError("This tokenizer does not have language token configured")
216 
217             return self.to_language_token(self.language)
218 
219         def to_language_token(self, language):
220             if token := self.special_tokens.get(f"<|{language}|>", None):
221                 return token
222 
223             raise KeyError(f"Language {language} not found in tokenizer.")
224 
225         @cached_property
226         def all_language_tokens(self) -> Tuple[int]:
227             result = []
228             for token, token_id in self.special_tokens.items():
229                 if token.strip("<|>") in LANGUAGES:
230                     result.append(token_id)
231             return tuple(result)[: self.num_languages]
232 
233         @cached_property
234         def all_language_codes(self) -> Tuple[str]:
235             return tuple(self.decode([_l]).strip("<|>") for _l in self.all_language_tokens)
236 
237         @cached_property
238         def sot_sequence_including_notimestamps(self) -> Tuple[int]:
239             return tuple(list(self.sot_sequence) + [self.no_timestamps])
240 
241         @cached_property
242         def non_speech_tokens(self) -> Tuple[int]:
243             """
244             Returns the list of tokens to suppress in order to avoid any speaker tags or non-speech
```

```
        annotations, to prevent sampling texts that are not actually spoken in the audio, e.g.

        - ♪♪♪
        - ( SPEAKING FOREIGN LANGUAGE )
        - [DAVID] Hey there,

        keeping basic punctuations like commas, periods, question marks, exclamation points, etc.
        """
        symbols = list('"#()*+/:;<=>@[\\]^_`{|}~「」『』 ')
        symbols += (
            "<< >> <<< >>> -- --- -( -[ (' (\" (( )) ((( ))) [[ ]] {{ }} ♪♪ ♪♪♪".split()
        )

        # symbols that may be a single token or multiple tokens depending on the tokenizer.
        # In case they're multiple tokens, suppress the first token, which is safe because:
        # These are between U+2640 and U+267F miscellaneous symbols that are okay to suppress
        # in generations, and in the 3-byte UTF-8 representation they share the first two bytes.
        miscellaneous = set("♩♪♫♬♭♮♯")
        assert all(0x2640 <= ord(c) <= 0x267F for c in miscellaneous)

        # allow hyphens "-" and single quotes "'" between words, but not at the beginning of a word
        result = {self.encoding.encode(" -")[0], self.encoding.encode(" '")[0]}
        for symbol in symbols + list(miscellaneous):
            for tokens in [
                self.encoding.encode(symbol),
                self.encoding.encode(" " + symbol),
            ]:
                if len(tokens) == 1 or symbol in miscellaneous:
                    result.add(tokens[0])

        return tuple(sorted(result))

    def split_to_word_tokens(self, tokens: List[int]):
        if self.language in {"zh", "ja", "th", "lo", "my", "yue"}:
            # These languages don't typically use spaces, so it is difficult to split words
            # without morpheme analysis. Here, we instead split words at any
            # position where the tokens are decoded as valid unicode points
            return self.split_tokens_on_unicode(tokens)
```

```
283
284             return self.split_tokens_on_spaces(tokens)
285
286         def split_tokens_on_unicode(self, tokens: List[int]):
287             decoded_full = self.decode_with_timestamps(tokens)
288             replacement_char = "\ufffd"
289
290             words = []
291             word_tokens = []
292             current_tokens = []
293             unicode_offset = 0
294
295             for token in tokens:
296                 current_tokens.append(token)
297                 decoded = self.decode_with_timestamps(current_tokens)
298
299                 if (
300                     replacement_char not in decoded
301                     or decoded_full[unicode_offset + decoded.index(replacement_char)]
302                     == replacement_char
303                 ):
304                     words.append(decoded)
305                     word_tokens.append(current_tokens)
306                     current_tokens = []
307                     unicode_offset += len(decoded)
308
309             return words, word_tokens
310
311         def split_tokens_on_spaces(self, tokens: List[int]):
312             subwords, subword_tokens_list = self.split_tokens_on_unicode(tokens)
313             words = []
314             word_tokens = []
315
316             for subword, subword_tokens in zip(subwords, subword_tokens_list):
317                 special = subword_tokens[0] >= self.eot
318                 with_space = subword.startswith(" ")
319                 punctuation = subword.strip() in string.punctuation
320                 if special or with_space or punctuation or len(words) == 0:
321                     words.append(subword)
```

```
322                     word_tokens.append(subword_tokens)
323                 else:
324                     words[-1] = words[-1] + subword
325                     word_tokens[-1].extend(subword_tokens)
326 
327             return words, word_tokens
328 
329 
330     @lru_cache(maxsize=None)
331     def get_encoding(name: str = "gpt2", num_languages: int = 99):
332         vocab_path = os.path.join(os.path.dirname(__file__), "assets", f"{name}.tiktoken")
333         ranks = {
334             base64.b64decode(token): int(rank)
335             for token, rank in (line.split() for line in open(vocab_path) if line)
336         }
337         n_vocab = len(ranks)
338         special_tokens = {}
339 
340         specials = [
341             "<|endoftext|>",
342             "<|startoftranscript|>",
343             *[f"<|{lang}|>" for lang in list(LANGUAGES.keys())[:num_languages]],
344             "<|translate|>",
345             "<|transcribe|>",
346             "<|startoflm|>",
347             "<|startofprev|>",
348             "<|nospeech|>",
349             "<|notimestamps|>",
350             *[f"<|{i * 0.02:.2f}|>" for i in range(1501)],
351         ]
352 
353         for token in specials:
354             special_tokens[token] = n_vocab
355             n_vocab += 1
356 
357         return tiktoken.Encoding(
358             name=os.path.basename(vocab_path),
359             explicit_n_vocab=n_vocab,
```

```
            pat_str=r"""'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+""",
            mergeable_ranks=ranks,
            special_tokens=special_tokens,
        )


    @lru_cache(maxsize=None)
    def get_tokenizer(
        multilingual: bool,
        *,
        num_languages: int = 99,
        language: Optional[str] = None,
        task: Optional[str] = None,  # Literal["transcribe", "translate", None]
    ) -> Tokenizer:
        if language is not None:
            language = language.lower()
            if language not in LANGUAGES:
                if language in TO_LANGUAGE_CODE:
                    language = TO_LANGUAGE_CODE[language]
                else:
                    raise ValueError(f"Unsupported language: {language}")

        if multilingual:
            encoding_name = "multilingual"
            language = language or "en"
            task = task or "transcribe"
        else:
            encoding_name = "gpt2"
            language = None
            task = None

        encoding = get_encoding(name=encoding_name, num_languages=num_languages)

        return Tokenizer(
            encoding=encoding, num_languages=num_languages, language=language, task=task
        )
```